



Materia: SEMINARIO FINAL DE INGENIERIA EN SOFTWARE

Profesor: JORGE HUMBERTO CASSI

Alumno:

Victor Emmanuel Brizzi

Córdoba, 26/04/2026

INDICE

Resumen	3
Abstract.....	4
Titulo	5
Introducción.....	5
Antecedentes	5
Descripción del área problemática.....	6
Justificación	7
Objetivo General Del Proyecto.	7
Objetivos Específicos del Proyecto	8
Marco Teórico Referencial.....	8
Dominio del problema	8
Tecnologías de la Información y la Comunicación (TIC)	9
Competencia	10
Diseño Metodológico	10
Relevamiento	13
Relevamiento estructural	13
Relevamiento funcional	14
Relevamiento de documentación	15
Procesos de negocio	16
Diagnóstico y propuesta	17
Nombre del proceso: Evaluación de calidad no funcional del software.....	17
Objetivos, límites y alcances del prototipo.....	18
Objetivo del prototipo	18
Límites	18
Alcances.....	18
Descripción del Sistema	19
Requerimientos funcionales.....	19
Requerimientos no funcionales.....	20
Diagrama de casos de uso	21

	2
Descripción de casos de uso	22
Diagrama de secuencia	28
Estructura de Datos	29
Prototipos de Interfaces del Sistema	31
Diagrama de despliegue	36
Seguridad	36
Análisis De Costos	40
Análisis De Riesgos	44
Conclusiones	50
Referencias	52
Anexo A: Reporte de análisis estático de código (SonarQube)	54
Anexo B: Reporte de escaneo de vulnerabilidades (OWASP ZAP)	55
Anexo C: Resultados de pruebas de carga y rendimiento (k6)	56

Resumen

El presente trabajo final de grado consistió en el desarrollo de un framework automatizado para la evaluación continua de requerimientos no funcionales en aplicaciones de software. La propuesta surgió de la necesidad de superar la validación fragmentaria, tardía y manual de atributos críticos de calidad como la mantenibilidad, seguridad y rendimiento. Para resolver esta problemática, se diseñó e implementó un sistema que integró y orquestó la ejecución de herramientas de análisis estático, escaneo de vulnerabilidades web y pruebas de carga mediante contenedores aislados. Asimismo, se desarrolló un algoritmo de normalización matemática que consolidó métricas heterogéneas en un indicador porcentual de puntuación unificado, y se almacenó el historial de evaluaciones en una base de datos. La aplicabilidad de la solución se validó a través de un caso de estudio práctico, el cual demostró que la automatización de estos análisis actuó como un control de calidad continuo en etapas tempranas del ciclo de vida del software. Se concluyó que el framework desarrollado redujo la carga operativa en pruebas de regresión, eliminó la subjetividad en la interpretación de los reportes y proporcionó visibilidad objetiva sobre la calidad no funcional del software antes de su puesta en producción.

Palabras clave: calidad de software, pruebas automatizadas, requerimientos no funcionales, integración continua.

Abstract

This final grade project consisted of the development of an automated framework for the continuous evaluation of non-functional requirements in software applications. The proposal arose from the need to overcome the fragmented, delayed, and manual validation of critical quality attributes such as maintainability, security, and performance. To resolve this problem, a system was designed and implemented to integrate and orchestrate the execution of static analysis, web vulnerability scanning, and load testing tools through isolated containers. Likewise, a mathematical normalization algorithm was developed to consolidate heterogeneous metrics into a unified percentage scoring indicator, and the evaluation history was stored in a database. The applicability of the solution was validated through a practical case study, which demonstrated that the automation of these analyses acted as a continuous quality control in early stages of the software development life cycle. It was concluded that the developed framework reduced the operational workload in regression testing, eliminated subjectivity in report interpretation, and provided objective visibility on the non-functional quality of the software before its production deployment.

Keywords: software quality, automated testing, non-functional requirements, continuous integration.

Título

Modelo Automatizado De Evaluación Continua De Requerimientos No Funcionales

Introducción

El presente trabajo tuvo como objetivo desarrollar un framework automatizado para la evaluación de requerimientos no funcionales en aplicaciones de software, mediante la integración de herramientas de análisis y la utilización de métricas objetivas. La propuesta buscó generar indicadores cuantificables y reportes estandarizados que permitieran obtener una visión integral del estado de calidad de un sistema, lo que facilitó la toma de decisiones dentro del ciclo de desarrollo.

Antecedentes

En el desarrollo de software moderno, la calidad de los sistemas no se limita únicamente al cumplimiento de requerimientos funcionales, sino que también depende de atributos no funcionales como el rendimiento, la seguridad, la usabilidad y la mantenibilidad. Estos atributos han sido formalmente definidos en estándares como ISO/IEC 25010, el cual establece un modelo de calidad que permite evaluar diferentes características de los productos de software.

Sin embargo, los requerimientos no funcionales presentan dificultades particulares en su especificación y validación. Según Sommerville (2011), estos requerimientos suelen ser más difíciles de verificar que los requerimientos funcionales, lo que genera ambigüedad en su interpretación y dificulta su evaluación a lo largo del ciclo de desarrollo.

En la práctica, la evaluación de estos atributos suele realizarse en etapas tardías del desarrollo o incluso en producción, lo que incrementa el riesgo de fallos y retrabajo. En este sentido, Forsgren, Humble y Kim (2018) destacan que la integración de prácticas de calidad dentro del flujo de desarrollo permite mejorar la estabilidad y el desempeño de los sistemas, reduciendo problemas detectados en etapas finales.

Por otra parte, la evaluación de los requerimientos no funcionales suele apoyarse en herramientas especializadas que abordan aspectos específicos de forma aislada, como

el rendimiento, la seguridad o la calidad del código. Esta fragmentación dificulta la obtención de una visión integral del estado de calidad de una aplicación y limita la capacidad de tomar decisiones basadas en métricas unificadas.

Asimismo, enfoques modernos como los propuestos en Site Reliability Engineering enfatizan la importancia de la medición continua y el uso de métricas para garantizar la confiabilidad de los sistemas, destacando la necesidad de contar con mecanismos sistemáticos de medición a lo largo del tiempo (Beyer, Jones, Petoff y Murphy, 2016).

Descripción del área problemática

En el contexto actual del desarrollo de software, los equipos técnicos requieren contar con información confiable y oportuna sobre la calidad de sus aplicaciones más allá del correcto funcionamiento funcional. En particular, resulta fundamental evaluar atributos no funcionales como el rendimiento, la seguridad y la mantenibilidad, debido a su impacto directo en el comportamiento del sistema en entornos productivos.

No obstante, en entornos reales de desarrollo, la evaluación de estos atributos suele integrarse de manera limitada dentro del ciclo de trabajo. En muchos casos, los equipos utilizan herramientas específicas de forma independiente, sin una estrategia común que permita unificar criterios o resultados.

Como consecuencia, la información obtenida a partir de estos análisis se presenta de forma dispersa, dificultando la interpretación global del estado de calidad de una aplicación. Esta situación obliga a los equipos a realizar análisis manuales y a basar sus decisiones en múltiples fuentes de información no integradas.

Adicionalmente, la falta de mecanismos sistemáticos para registrar y comparar resultados a lo largo del tiempo limita la capacidad de identificar tendencias, mejoras o regresiones en la calidad del software.

Esta situación evidencia una limitación en la forma en que se evalúan los requerimientos no funcionales en entornos reales de desarrollo.

Justificación

El presente proyecto surgió a partir de la necesidad de mejorar la evaluación de los requerimientos no funcionales en el desarrollo de software. En particular, se identificó la falta de un enfoque integrado que permitiera interpretar de manera unificada los resultados obtenidos a partir de distintas herramientas de análisis.

En este sentido, el proyecto buscó desarrollar un enfoque automatizado, continuo y estandarizado que permitiera integrar la evaluación de estos atributos dentro del ciclo de desarrollo, facilitando la detección temprana de problemas, reduciendo el riesgo de fallos en producción y mejorando la calidad general del software.

Desde el punto de vista tecnológico, la propuesta introdujo un modelo que no solo permitió medir distintos atributos de calidad, sino también interpretarlos mediante un sistema de scoring unificado y analizarlos a lo largo del tiempo. Esto representó un avance respecto de los enfoques tradicionales, que suelen centrarse en métricas aisladas sin considerar su evolución ni su impacto conjunto.

Asimismo, el valor del proyecto para las organizaciones radicó en la posibilidad de contar con información objetiva, centralizada y comparable sobre la calidad de sus sistemas, lo que permitió optimizar los procesos de desarrollo, mejorar la toma de decisiones y reducir costos asociados a la detección tardía de errores. En un contexto donde los sistemas informáticos resultan críticos para la operación de las organizaciones, disponer de este tipo de herramientas adquirió una relevancia significativa.

Finalmente, el proyecto introdujo una mejora a nivel de procesos, al transformar la evaluación de los requerimientos no funcionales en una actividad continua, automatizada y basada en métricas. Esto permitió pasar de un enfoque reactivo, donde los problemas se detectaban en etapas tardías, a un enfoque proactivo, donde la calidad fue monitoreada de forma constante a lo largo del ciclo de vida del software.

Objetivo General Del Proyecto.

Desarrollar un framework automatizado para la evaluación de requerimientos no funcionales en aplicaciones de software, basado en la ejecución de pruebas y el uso de

métricas objetivas, que permita generar reportes estandarizados con indicadores cuantificables y cuya aplicabilidad sea validada mediante un caso de estudio.

Objetivos Específicos del Proyecto

- Analizar la evaluación de requerimientos no funcionales en aplicaciones de software para definir los atributos de calidad a sistematizar.
- Examinar la ejecución de pruebas y el uso de métricas objetivas en diversas herramientas de testing para determinar la viabilidad de su integración.
- Construir el framework automatizado que integre las herramientas seleccionadas para generar reportes estandarizados con indicadores cuantificables.
- Validar la aplicabilidad del framework propuesto mediante un caso de estudio empírico.

Marco Teórico Referencial

Dominio del problema

En el desarrollo de software moderno, los requerimientos no funcionales (NFR) constituyen atributos de calidad que determinan el comportamiento de un sistema, más allá de las funcionalidades que este ofrece. De acuerdo con el estándar ISO/IEC 25010, la calidad del software se define a partir de características como el rendimiento, la seguridad y la mantenibilidad, entre otras.

Diversos autores coinciden en que los requerimientos no funcionales presentan dificultades en su especificación y medición objetiva, lo que genera ambigüedad durante su implementación (Chung, Nixon, Yu y Mylopoulos, 2000). Esto se debe a su carácter transversal y a su dependencia del contexto en el que se aplica el sistema.

En el ámbito industrial, la evaluación de estos atributos suele realizarse mediante herramientas especializadas que operan de forma independiente. Por ejemplo, la seguridad se evalúa a través de análisis de vulnerabilidades, mientras que el rendimiento se

mide mediante pruebas de carga. Sin embargo, estos enfoques carecen de un modelo unificado que permita integrar y analizar los resultados de manera conjunta (Bass, Clements y Kazman, 2012).

Tecnologías de la Información y la Comunicación (TIC)

Para la implementación del framework propuesto se consideran tecnologías y herramientas que permiten la automatización de pruebas, la integración de resultados y la generación de reportes. A continuación, se presentan las principales tecnologías utilizadas, junto con su definición y referencia bibliográfica correspondiente.

SonarQube: Plataforma de análisis estático de código que permite evaluar la calidad del software mediante métricas como mantenibilidad, complejidad y duplicación del código (Campbell y Papapetrou, 2013).

OWASP ZAP: Herramienta de código abierto orientada a la detección de vulnerabilidades en aplicaciones web mediante pruebas automatizadas de seguridad (OWASP Foundation, 2023).

k6: Herramienta de pruebas de carga que permite evaluar el rendimiento de aplicaciones mediante la simulación de usuarios concurrentes y el análisis del comportamiento del sistema bajo condiciones de estrés (Grafana Labs, 2023).

Node.js: Entorno de ejecución basado en JavaScript que permite desarrollar aplicaciones escalables mediante un modelo asíncronico orientado a eventos (Node.js Foundation, 2023).

API REST: Estilo arquitectónico que permite la comunicación entre sistemas a través del protocolo HTTP, utilizando recursos identificados mediante URLs y formatos de intercambio de datos como JSON (Fielding, 2000).

Docker: Tecnología de contenedorización que permite empaquetar aplicaciones junto con sus dependencias, garantizando la portabilidad, el aislamiento y la reproducibilidad de los entornos de ejecución (Merkel, 2014).

Chart.js: Biblioteca de JavaScript que permite representar datos mediante gráficos interactivos y dinámicos en aplicaciones web (Chart.js, 2023).

Puppeteer: Herramienta que permite controlar un navegador de forma programática para generar documentos en formato PDF a partir de contenido HTML (Google Developers, 2023).

Base de datos relacional: Sistema de almacenamiento de información que organiza los datos en tablas relacionadas entre sí, permitiendo su consulta, gestión e integridad mediante el uso de lenguajes como SQL (Date, 2004).

Competencia

En la actualidad, existen diversas herramientas que permiten evaluar aspectos específicos de la calidad del software de manera independiente.

Por ejemplo, SonarQube se orienta al análisis de la calidad del código, proporcionando métricas relacionadas con la mantenibilidad y complejidad (Campbell y Papapetrou, 2013), mientras que OWASP ZAP se especializa en la detección de vulnerabilidades de seguridad en aplicaciones web mediante pruebas automatizadas (OWASP Foundation, 2023). Por su parte, k6 permite realizar pruebas de rendimiento a través de la simulación de carga sobre sistemas, evaluando su comportamiento bajo condiciones de estrés (Grafana Labs, 2023).

Asimismo, herramientas como Google Lighthouse permiten generar reportes automatizados con indicadores de desempeño, accesibilidad y buenas prácticas, aunque se encuentran limitadas al análisis de aplicaciones web (Google, 2023).

Diseño Metodológico

Para estructurar el trabajo, el conjunto de herramientas metodológicas y de desarrollo aplicadas en este proyecto inició con la adopción de los lineamientos del Lenguaje Unificado de Modelado (UML), siguiendo un enfoque basado en el paradigma de orientación a objetos. Esta elección metodológica permitió representar de manera estructurada los distintos componentes del sistema, sus interacciones y la arquitectura general propuesta. La aplicación de UML facilitó enormemente las fases de análisis, diseño y docu-

mentación del framework, permitiendo una mejor comprensión del funcionamiento interno y asegurando una trazabilidad coherente entre los requerimientos no funcionales definidos y la solución técnica implementada.

A nivel puramente técnico, el proyecto requirió la integración de diversas tecnologías y lenguajes (previamente contemplados en el marco teórico), los cuales fueron seleccionados estratégicamente por su capacidad para automatizar, escalar y estandarizar la evaluación de calidad de software.

Como pilar central de la arquitectura, se eligió Node.js para la capa de backend, actuando como el motor de orquestación encargado de gestionar la ejecución de pruebas, procesar los resultados unificados y coordinar la lógica principal del framework. Para asegurar que los componentes del sistema interactúen de forma ágil, la comunicación interna y la conexión con herramientas externas se estructuró mediante APIs REST. Esta decisión técnica resultó crucial, ya que permitió estandarizar el intercambio de datos utilizando el formato JSON.

Dada la necesidad de ejecutar las evaluaciones de manera reproducible y evitar conflictos de dependencias, se implementó Docker. Esta tecnología de contenerización permitió encapsular cada herramienta en un entorno de ejecución aislado, garantizando que el framework mantenga su integridad sin depender de la infraestructura subyacente.

En cuanto al núcleo de evaluación de los requerimientos no funcionales, el sistema integró de manera orquestada tres utilidades especializadas, cada una elegida para atacar una categoría crítica de calidad:

- SonarQube, encargado de realizar el análisis estático para medir la mantenibilidad y la calidad estructural del código fuente.
- OWASP ZAP, utilizado para escanear dinámicamente el aplicativo en busca de vulnerabilidades de seguridad.
- k6, seleccionado para someter al sistema a pruebas de carga, obteniendo así métricas objetivas de rendimiento operativo.

Finalmente, para consolidar el objetivo de generar reportes cuantificables, el sistema unifica las métricas arrojadas por las herramientas mencionadas y las visualiza dinámicamente mediante la biblioteca Chart.js. Como complemento final, la arquitectura integró Puppeteer para la generación automatizada de informes en formato PDF a partir de los datos procesados, facilitando enormemente su interpretación, análisis y distribución.

Recolección de datos

La recolección de datos se llevó a cabo mediante el análisis de información proveniente de distintas fuentes, tanto teóricas como prácticas, con el objetivo de comprender el dominio del problema y validar el funcionamiento del framework propuesto.

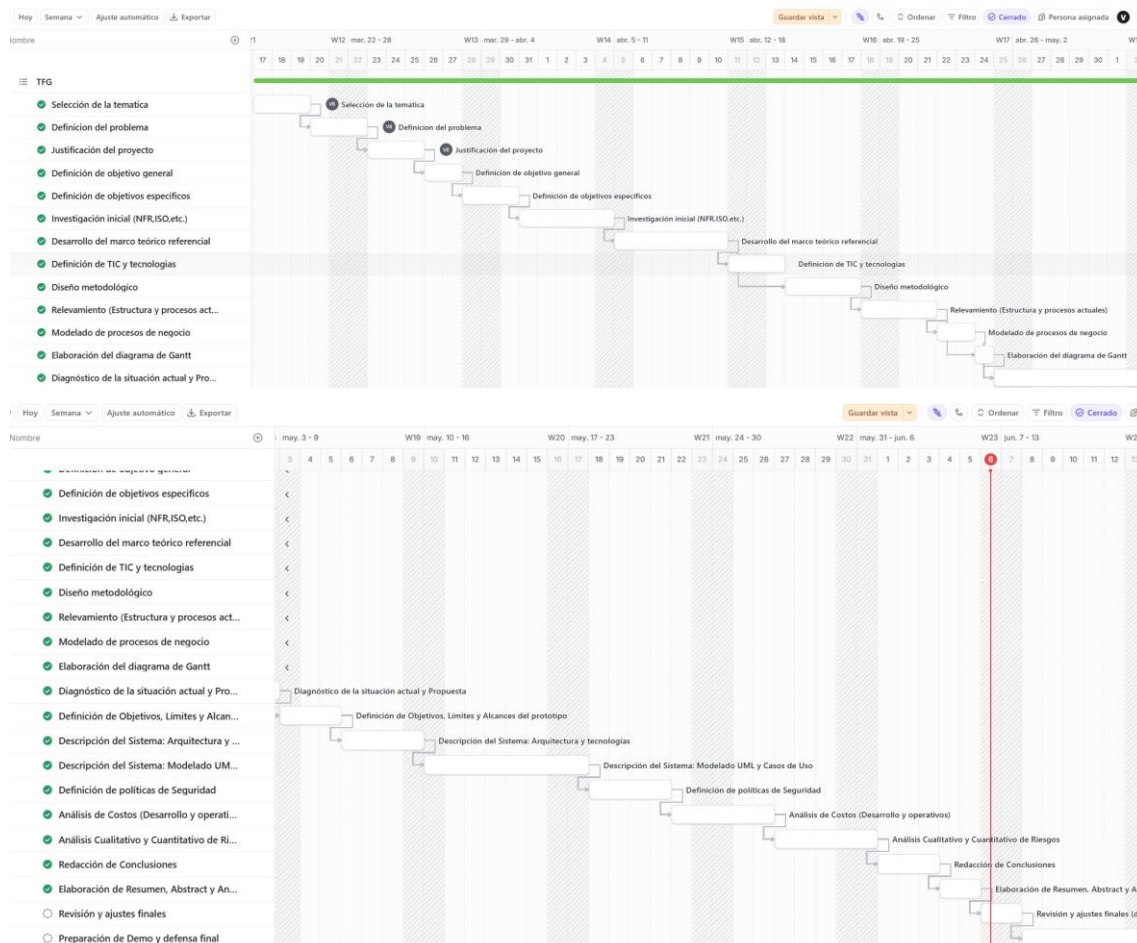
En primer lugar, se realizó una revisión bibliográfica de estándares y modelos de calidad de software, como ISO/IEC 25010, así como de literatura académica relacionada con requerimientos no funcionales y métricas de software.

En segundo lugar, se aplicó una técnica de análisis comparativo (benchmarking) de herramientas existentes, evaluando soluciones actuales del mercado que permiten medir aspectos específicos de la calidad del software, tales como seguridad, rendimiento y mantenibilidad, mediante la definición de criterios de evaluación que permitieron identificar sus capacidades, limitaciones y posibilidades de integración.

Para la planificación del desarrollo del proyecto se elaboró un diagrama de Gantt, en el cual se definieron las actividades correspondientes a la primera etapa del trabajo, así como su secuencia y duración estimada.

La planificación permitió organizar el trabajo de manera estructurada, asegurando el cumplimiento de los plazos establecidos para la presente entrega y la correcta elaboración de los contenidos requeridos.

El plan de actividades se detalló en el siguiente diagrama de Gantt, en el cual se representaron las tareas desarrolladas, su secuencia temporal y las relaciones de dependencia entre ellas.



Relevamiento

Relevamiento estructural

El presente proyecto se desarrolló en el contexto de una organización modelada, representativa de empresas dedicadas al desarrollo de software. Por este motivo, no se puede brindar una localización física ni infraestructura tecnológica.

Relevamiento funcional

Dado que se trató de una organización modelada, no se definió una estructura jerárquica formal, sino que se identificaron los roles y funciones involucrados en el proceso de evaluación de calidad del software.

Roles identificados:

1. Desarrollador: responsable de implementar funcionalidades en el sistema.
2. QA / Tester: encargado de validar el correcto funcionamiento del sistema mediante pruebas.
3. DevOps / SRE: responsable de la integración, despliegue y monitoreo del sistema.
4. Responsable técnico / arquitecto: encargado de tomar decisiones sobre la calidad y arquitectura del sistema.

Funciones:

- Desarrollo de funcionalidades del sistema, realizado por el Desarrollador.
- Ejecución de pruebas funcionales y no funcionales, llevada a cabo por el QA / Tester.
- Integración continua y despliegue de aplicaciones, responsabilidad del DevOps / SRE.
- Evaluación de métricas de calidad del software, realizada por el QA / Tester y el Responsable técnico.
- Toma de decisiones en base a resultados de calidad, a cargo del Responsable técnico / arquitecto.

Procesos relevados:

Nombre del proceso: Evaluación de calidad no funcional del software

Roles: Desarrollador, QA, DevOps, Responsable técnico

Pasos:

- El desarrollador implementa cambios en el código fuente y los integra en el repositorio.
- Se despliega la aplicación en un entorno de testing o staging.
- El equipo de QA ejecuta pruebas funcionales sobre la aplicación, enfocándose principalmente en la validación de funcionalidades.
- De manera aislada, algunos miembros del equipo ejecutan herramientas específicas para evaluar aspectos particulares del sistema, como análisis de código, pruebas de seguridad y pruebas de rendimiento.
- Como resultado de estas ejecuciones, se generan artefactos de salida tales como reportes de calidad de código (SonarQube), reportes de vulnerabilidades (OWASP ZAP), resultados de pruebas de rendimiento (k6) y logs de ejecución, los cuales contienen métricas, eventos y resultados asociados a cada evaluación realizada.
- El Responsable Técnico (o el equipo) revisa manualmente los distintos reportes por separado para interpretar el estado del sistema y tomar una decisión sobre la calidad de la versión.

Relevamiento de documentación

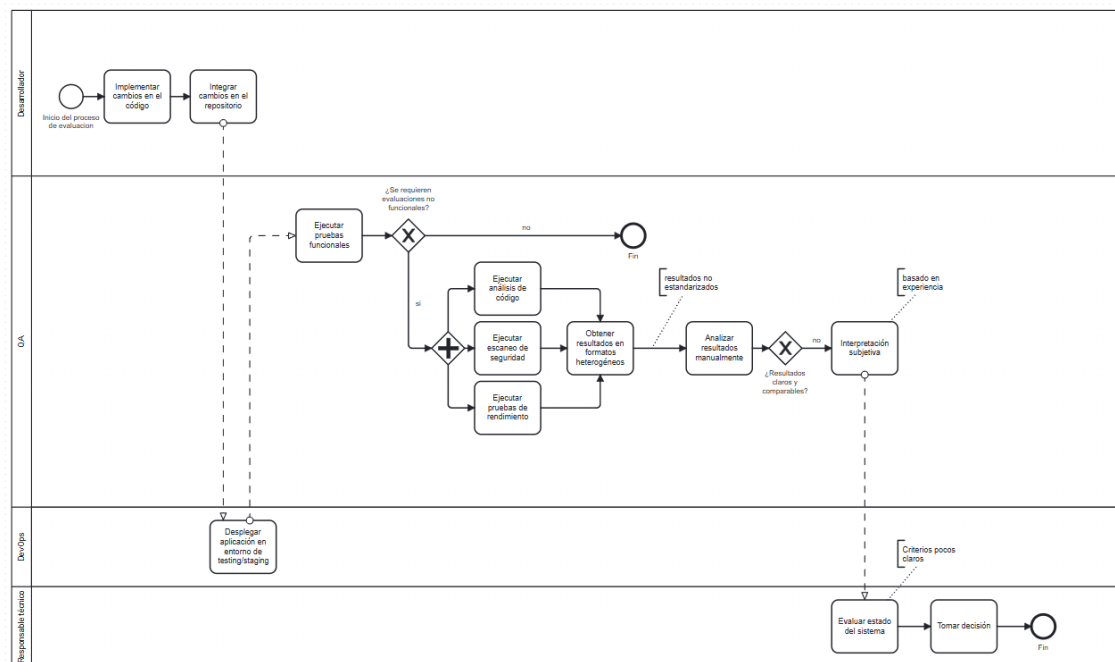
Se identifican los siguientes artefactos generados durante el proceso:

- Reportes individuales de herramientas de análisis de código: Documentos generados por herramientas de análisis estático (como SonarQube) que detallan métricas de mantenibilidad, deuda técnica, code smells y cobertura de código de la aplicación. (Ver Anexo A: Ejemplo de reporte de análisis de código).
- Resultados de escaneos de seguridad: Informes arrojados por herramientas de análisis dinámico (como OWASP ZAP) que listan las vulnerabilidades encontradas en el aplicativo durante las pruebas, clasificándolas según su nivel de criticidad (alta, media, baja). (Ver Anexo B: Ejemplo de reporte de vulnerabilidades).
- Resultados de pruebas de rendimiento: Archivos de salida (generados por herramientas como k6) que contienen datos estadísticos sobre tiempos de respuesta,

latencia y tasas de error del sistema bajo distintas cargas de usuarios concurrentes.
(Ver Anexo C: Ejemplo de resultados de rendimiento).

Procesos de negocio

El análisis de la evaluación actual de requerimientos no funcionales dentro del proceso de negocio modelado reflejó la situación de las organizaciones de desarrollo de software, evidenciando la ejecución aislada de herramientas, la falta de estandarización de resultados y la dependencia de análisis manual para la toma de decisiones.



Diagnóstico y propuesta

Nombre del proceso: Evaluación de calidad no funcional del software

Tabla 1: Diagnóstico de problemas y causas

Problemas	Causas
La evaluación de los requerimientos no funcionales se realiza de manera fragmentada, utilizando herramientas independientes, lo que dificulta obtener una visión integral del estado de calidad del sistema.	- Uso de herramientas especializadas sin integración entre sí. - Ausencia de un modelo unificado para consolidar resultados.
Los resultados obtenidos presentan formatos heterogéneos, lo que complica su interpretación y comparación.	- Cada herramienta genera salidas con estructuras y métricas propias. - Falta de estandarización en la presentación de resultados.
El análisis de los resultados se realiza de forma manual, generando dependencia de la interpretación del equipo técnico.	- Ausencia de mecanismos automáticos de procesamiento y análisis. - Falta de indicadores cuantificables comunes.
No se dispone de un seguimiento histórico que permita comparar la evolución de la calidad del sistema.	- Los resultados no se almacenan de forma estructurada. - No existen herramientas que permitan versionar y comparar ejecuciones.
La toma de decisiones se basa en criterios subjetivos, lo que puede generar inconsistencias en la evaluación del sistema.	- Falta de métricas unificadas y objetivas. - Dependencia de la experiencia individual de los evaluadores.

Se propuso el desarrollo de un framework automatizado para la evaluación de requerimientos no funcionales que permitiera integrar la ejecución de pruebas, el procesamiento de resultados y la generación de indicadores cuantificables en un único sistema.

El framework centralizó la ejecución de herramientas especializadas para evaluar distintos atributos de calidad, tales como mantenibilidad, seguridad y rendimiento, permitiendo unificar sus resultados mediante un modelo de datos común.

Asimismo, incorporó un mecanismo de normalización y scoring que permitió transformar los resultados obtenidos en indicadores comparables, facilitando la interpretación del estado de calidad del sistema.

El sistema también permitió almacenar el historial de ejecuciones, posibilitando el análisis de la evolución de la calidad a lo largo del tiempo y la detección de mejoras o regresiones entre distintas versiones.

Finalmente, se generaron reportes estandarizados que integraron los resultados obtenidos, permitiendo a los equipos técnicos y responsables de decisión contar con información clara, objetiva y centralizada para la evaluación del sistema.

Objetivos, límites y alcances del prototipo

Objetivo del prototipo

Desarrollar un prototipo funcional que permita automatizar la ejecución de pruebas de requerimientos no funcionales, procesar e integrar sus resultados, y generar reportes estandarizados con indicadores cuantificables de calidad.

Límites

Desde la ejecución de evaluaciones automatizadas de requerimientos no funcionales sobre una aplicación de software, hasta la generación de reportes estandarizados con indicadores cuantificables para la toma de decisiones.

Alcances

- Ejecución de evaluaciones automatizadas de requerimientos no funcionales en las categorías de mantenibilidad, seguridad y rendimiento.
- Integración y normalización de resultados provenientes de herramientas externas.
- Procesamiento de métricas y cálculo de indicadores de calidad.
- Generación de reportes estandarizados y visualización de resultados mediante dashboards interactivos.
- Almacenamiento y consulta del historial de ejecuciones

Descripción del Sistema

Requerimientos funcionales

- El sistema deberá permitir:
- RF01: Registrar aplicaciones de software que serán evaluadas por el framework.
- RF02: Configurar evaluaciones de requerimientos no funcionales seleccionando las categorías de mantenibilidad, seguridad y rendimiento.
- RF03: Definir parámetros de ejecución para las pruebas automatizadas, tales como URL objetivo, repositorio, entorno, duración o cantidad de usuarios virtuales según corresponda.
- RF04: Seleccionar las herramientas externas que serán utilizadas en cada evaluación según la categoría de requerimiento no funcional.
- RF05: Ejecutar pruebas automatizadas utilizando herramientas externas de análisis según la configuración definida.
- RF06: Integrar los resultados obtenidos desde herramientas externas en un formato común de procesamiento.
- RF07: Normalizar las métricas obtenidas para permitir su comparación entre distintas categorías de evaluación.
- RF08: Calcular puntajes individuales por categoría de requerimiento no funcional evaluado.
- RF09: Calcular un puntaje global de calidad no funcional a partir de los resultados obtenidos.
- RF10: Almacenar los resultados de cada ejecución junto con su estado (pendiente, en proceso, finalizada o con error).
- RF11: Visualizar los resultados obtenidos mediante dashboards con indicadores, gráficos y detalle de hallazgos.
- RF12: Generar reportes estandarizados con los resultados de la evaluación y permitir su exportación en formato PDF.

- RF13: Consultar el historial de evaluaciones realizadas sobre una aplicación.
- RF14: Comparar resultados entre distintas ejecuciones para identificar mejoras o regresiones en la calidad no funcional del sistema.

Requerimientos no funcionales

- RNF01: Mantenibilidad

El sistema deberá alcanzar un nivel de mantenibilidad equivalente a Rating A en herramientas de análisis estático de código, con una complejidad ciclomática promedio inferior a 10 por función, un nivel de duplicación de código inferior al 5% y ausencia de code smells de severidad alta.

- RNF02: Seguridad

El sistema deberá presentar cero vulnerabilidades de severidad alta y no más de tres vulnerabilidades de severidad media, medidas mediante herramientas automatizadas de análisis de seguridad sobre la aplicación desplegada.

- RNF03: Rendimiento

El sistema deberá mantener un tiempo de respuesta en el percentil 95 (p95) inferior a 500 milisegundos y una tasa de error inferior al 1%, bajo una carga simulada de 20 usuarios virtuales concurrentes durante un período de 60 segundos.

- RNF04: Evaluabilidad

El sistema deberá permitir la medición automatizada de sus atributos de calidad mediante la ejecución de herramientas externas integradas en el framework.

- RNF05: Reproducibilidad

El sistema deberá garantizar que una misma evaluación, ejecutada bajo las mismas condiciones, produzca resultados consistentes y comparables.

- RNF06: Trazabilidad

El sistema deberá registrar cada ejecución, incluyendo parámetros, métricas obtenidas y resultados, permitiendo su seguimiento y comparación en el tiempo.

- RNF07: Escalabilidad de evaluación

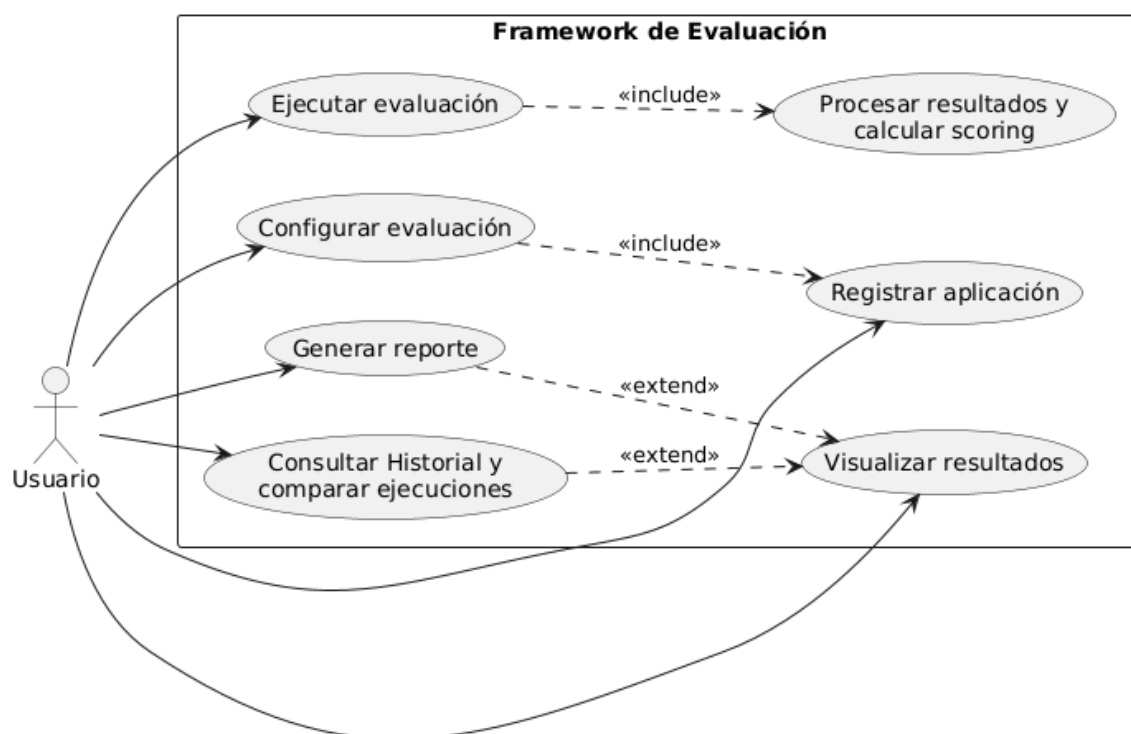
El sistema deberá permitir la ejecución de múltiples evaluaciones sin afectar significativamente los tiempos de procesamiento.

- RNF08: Extensibilidad

El sistema deberá permitir incorporar nuevas métricas o herramientas de evaluación sin modificar la arquitectura principal.

Diagrama de casos de uso

En la siguiente figura se exhibe el diagrama de casos de uso del framework propuesto, el cual ilustra las interacciones de los actores con las funcionalidades principales del sistema, desde la ejecución de las evaluaciones hasta la generación de los reportes.



*Descripción de casos de uso***Tabla 2: CU-01**

CU-01 Registrar aplicación		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF01	
Descripción	Permite registrar una aplicación de software que será evaluada por el framework.	
Precondición	El usuario debe contar con la información básica de la aplicación.	
Secuencia Normal	Paso	Acción
	1	El usuario ingresa al módulo de registro de aplicaciones.
	2	El sistema solicita los datos de la aplicación.
	3	El usuario carga nombre, descripción, URL objetivo y/o repositorio.
	4	El sistema valida los datos ingresados.
	5	El sistema registra la aplicación.
Postcondición	La aplicación queda registrada y disponible para configurar evaluaciones.	
Curso alternativo	Paso	Acción
	1	Si los datos ingresados son incompletos o inválidos, el sistema informa el error y solicita su corrección.
Frecuencia esperada	Media	
Importancia	Muy importante.	
Comentarios	-	

Tabla 3: CU-02

CU-02 Configurar evaluación		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF02, RF03, RF04	
Descripción	Permite definir los parámetros necesarios para ejecutar una evaluación de requerimientos no funcionales sobre una aplicación	
Precondición	La aplicación debe estar registrada en el sistema.	
Secuencia Normal	Paso	Acción
	1	El usuario accede al módulo de configuración de evaluación.
	2	El sistema muestra la lista de aplicaciones registradas.
	3	El usuario selecciona una aplicación.
	4	El sistema muestra las categorías disponibles (mantenibilidad, seguridad, rendimiento).
	5	El usuario selecciona una o más categorías de evaluación.
	6	El usuario define parámetros de ejecución (URL, repositorio, entorno, etc.).
	7	El usuario selecciona las herramientas externas a utilizar.
	8	El sistema valida la configuración ingresada.
	9	El sistema guarda la configuración de la evaluación.
Postcondición	La evaluación queda configurada y lista para ser ejecutada.	
Curso alternativo	Paso	Acción
	1	Si faltan parámetros obligatorios o la configuración es inválida, el sistema informa el error y solicita su corrección.
Frecuencia esperada	Media	
Importancia	Muy importante.	
Comentarios	-	

Tabla 4: CU-03 Ejecutar evaluación

CU-03 Ejecutar evaluación		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF05, RF10	
Descripción	Permite ejecutar una evaluación automatizada de requerimientos no funcionales sobre una aplicación previamente configurada.	
Precondición	Debe existir una evaluación configurada para una aplicación registrada.	
Secuencia Normal	Paso	Acción
	1	El usuario accede al módulo de evaluaciones.
	2	El sistema muestra las evaluaciones configuradas.
	3	El usuario selecciona una evaluación.
	4	El usuario inicia la ejecución.
	5	El sistema cambia el estado a “en proceso”.
	6	El sistema ejecuta las herramientas externas configuradas.
	7	El sistema registra el estado de la ejecución.
	8	El sistema finaliza la ejecución.
	9	El sistema actualiza el estado a “finalizada”.
Postcondición	La evaluación queda ejecutada y sus resultados disponibles para procesamiento.	
Curso alternativo	Paso	Acción
	1	Si ocurre un error en la ejecución de alguna herramienta, el sistema registra el estado como “error” e informa al usuario.
Frecuencia esperada	Alta.	
Importancia	Muy importante.	
Comentarios	-	

Tabla 5: CU-04 Procesar resultados y calcular scoring

CU-04		
Procesar resultados y calcular scoring		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF06, RF07, RF08, RF09	
Descripción	Permite integrar, normalizar y procesar los resultados obtenidos de las herramientas externas, calculando indicadores y puntajes de calidad.	
Precondición	Debe existir una evaluación finalizada con resultados disponibles.	
Secuencia Normal	Paso	Acción
	1	El sistema recupera los resultados generados por las herramientas externas.
	2	El sistema integra los resultados en un formato común.
	3	El sistema normaliza las métricas obtenidas.
	4	El sistema calcula los puntajes individuales por categoría.
	5	El sistema calcula el puntaje global de calidad.
	6	El sistema almacena los resultados procesados.
Postcondición	Los resultados quedan procesados, normalizados y almacenados con sus respectivos indicadores de calidad.	
Curso alternativo	Paso	Acción
	1	Si los resultados no pueden ser procesados correctamente, el sistema registra el error e informa al usuario.
Frecuencia esperada	Alta.	
Importancia	Muy importante.	
Comentarios	-	

Tabla 6: CU-05 Visualizar resultados

CU-05 Visualizar resultados		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF11	
Descripción	Permite visualizar los resultados de una evaluación mediante dashboards con indicadores, gráficos y detalle de hallazgos.	
Precondición	Debe existir una evaluación procesada con resultados disponibles.	
Secuencia Normal	Paso	Acción
	1	El usuario accede al módulo de resultados.
	2	El sistema muestra las evaluaciones disponibles.
	3	El usuario selecciona una evaluación.
	4	El sistema muestra el puntaje global de calidad.
	5	El sistema muestra los puntajes por categoría.
	6	El sistema presenta gráficos e indicadores en un dashboard.
	7	El usuario consulta el detalle de los hallazgos.
Postcondición	El usuario visualiza el estado de calidad de la aplicación evaluada.	
Curso alternativo	Paso	Acción
	1	Si no existen resultados disponibles, el sistema informa la situación.
Frecuencia esperada	Alta.	
Importancia	Muy importante.	
Comentarios	-	

Tabla 7: CU-06 Generar reporte

CU-06 Generar reporte		
Versión 1.0	Fecha:5/10/2026	
Objetivos asociados	RF12	
Descripción	Permite generar un reporte estandarizado con los resultados de la evaluación, incluyendo métricas, puntajes y observaciones relevantes.	
Precondición	Debe existir una evaluación procesada con resultados disponibles.	
Secuencia Normal	Paso	Acción
	1	El usuario accede al módulo de reportes.
	2	El sistema muestra las evaluaciones disponibles.
	3	El usuario selecciona una evaluación.
	4	El usuario solicita la generación del reporte.
	5	El sistema recopila métricas e indicadores.
	6	El sistema genera el reporte estandarizado.
	7	El sistema permite exportar el reporte en formato PDF.
Postcondición	El reporte queda generado y disponible para su descarga.	
Curso alternativo	Paso	Acción
	1	Si ocurre un error en la generación del reporte, el sistema informa la situación al usuario.
Frecuencia esperada	Media.	
Importancia	Muy importante.	
Comentarios	-	

Tabla 8: CU-07 Consultar historial y comparar ejecuciones

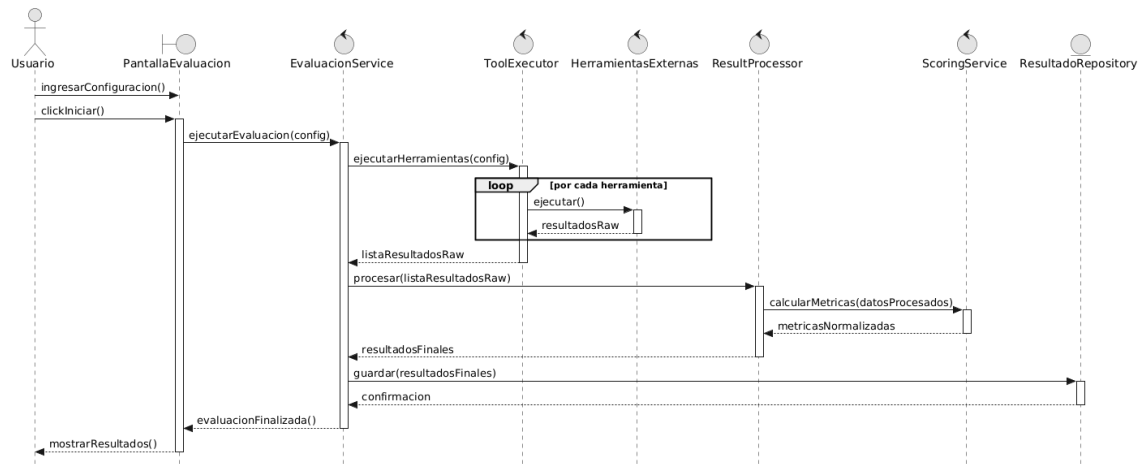
CU-07 Consultar historial y comparar ejecuciones	
Versión 1.0	Fecha:5/10/2026
Objetivos asociados	RF13, RF14

CU-07		
Consultar historial y comparar ejecuciones		
Descripción	Permite consultar evaluaciones anteriores y comparar sus resultados para analizar la evolución de la calidad del sistema.	
Precondición	Deben existir ejecuciones registradas en el sistema.	
Secuencia Normal	Paso	Acción
	1	El usuario accede al historial de evaluaciones.
	2	El sistema muestra las ejecuciones registradas.
	3	El usuario selecciona una o más evaluaciones.
	4	El sistema muestra los resultados de cada evaluación.
	5	El sistema compara métricas y puntajes.
	6	El sistema identifica mejoras o regresiones.
	7	El sistema presenta la comparación mediante gráficos o indicadores.
Postcondición	El usuario obtiene una comparación entre distintas ejecuciones.	
Curso alternativo	Paso	Acción
	1	Si no existen suficientes evaluaciones registradas para realizar la comparación, el sistema informa la situación al usuario.
Frecuencia esperada	Media.	
Importancia	Importante.	
Comentarios	-	

Diagrama de secuencia

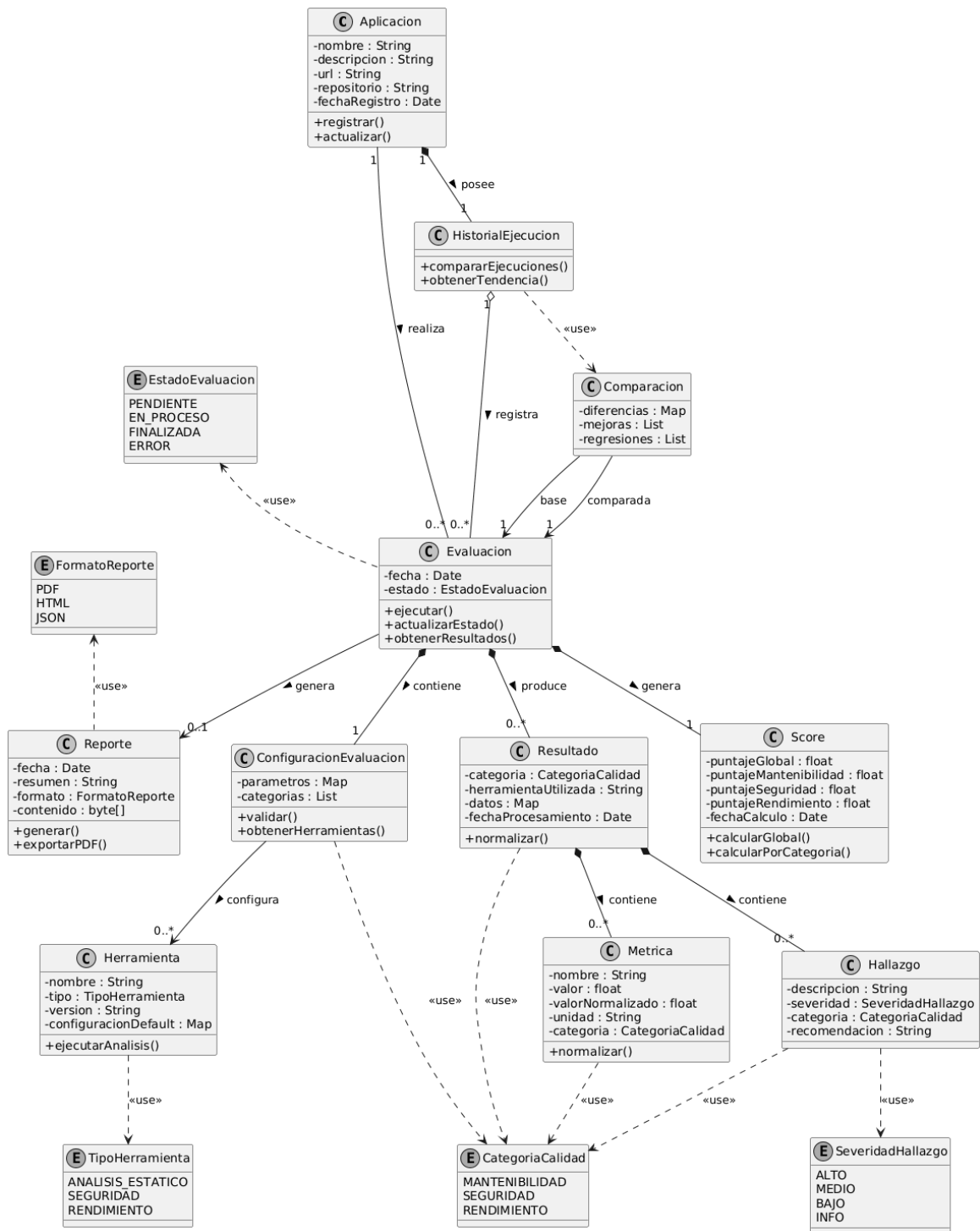
En la siguiente figura se exhibe el diagrama de secuencia correspondiente al proceso de ejecución de evaluaciones. El mismo detalla el flujo cronológico de mensajes

desde la interacción del usuario con la interfaz, pasando por los controladores de orquestación y herramientas externas, hasta el almacenamiento de las métricas en el repositorio.



Estructura de Datos

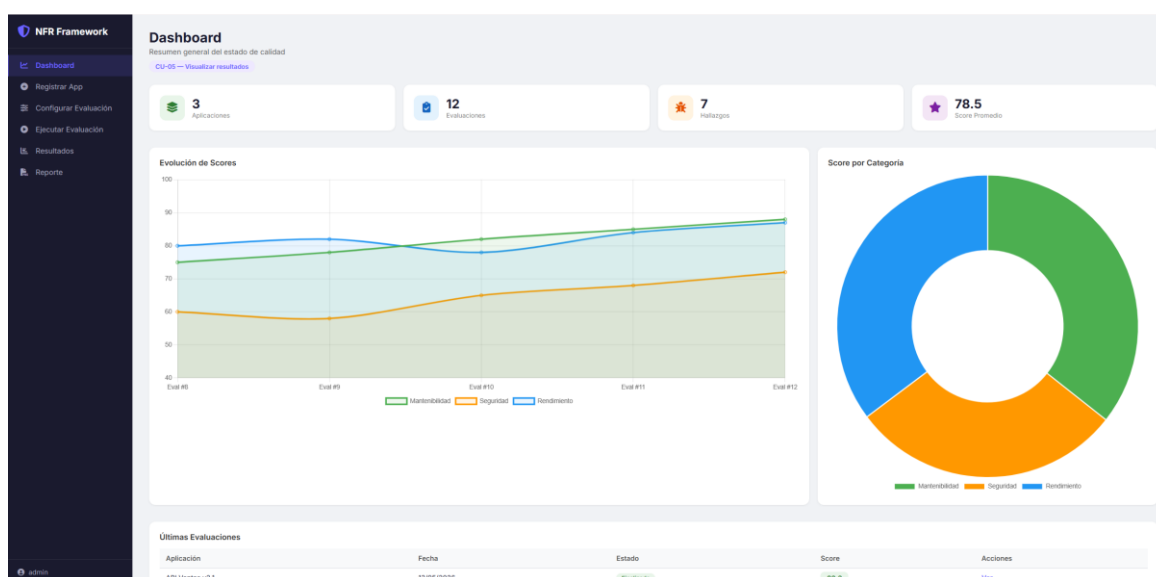
En la siguiente figura se exhibe el diagrama de clases que modela la estructura estática del framework. El diseño del dominio se centra en la entidad principal Evaluacion, la cual orquesta la configuración de las Herramientas de análisis y consolida la información técnica a través de las entidades Resultado, Metrica y Hallazgo. Finalmente, estos datos se procesan en un Score cuantitativo y se plasman en un Reporte, manteniendo siempre la trazabilidad en el HistorialEjecucion de cada Aplicacion.



Prototipos de Interfaces del Sistema

A continuación, se presentan las interfaces del sistema, las cuales representan el flujo de interacción del usuario con el framework, desde el registro de aplicaciones y la configuración de evaluaciones hasta la visualización de resultados y la generación de reportes. Cada pantalla se encuentra asociada a su caso de uso correspondiente, evidenciando el encadenamiento lógico del proceso.

1. Interfaz de Dashboard (Asociada al CU-05: Visualizar resultados)



Esta interfaz actúa como el punto de entrada principal del sistema y ofrece un resumen gerencial del estado de calidad del portfolio de aplicaciones. Su función es permitir al usuario visualizar de manera ágil, mediante gráficos interactivos, la evolución histórica de los scores. Además, expone un listado dinámico con los estados de las últimas evaluaciones ejecutadas, facilitando el acceso rápido al detalle de las mismas.

2. Interfaz de Registrar Aplicación (Asociada al CU-01: Registrar aplicación)

The screenshot displays the 'Registrar Aplicación' (Register Application) interface within the NFR Framework. On the left is a dark sidebar with navigation links: Dashboard, Registrar App (highlighted), Configurar Evaluación, Ejecutar Evaluación, Resultados, and Reporte. The main content area has a light blue background. At the top, it says 'Registrar Aplicación' followed by 'Registre una nueva aplicación para evaluar' and a breadcrumb 'CU-01 — Registrar aplicación'. Below this is a form with the following fields: 'Nombre de la aplicación *' with the value 'API Ventas'; 'Descripción' with the value 'API REST para gestión de ventas desarrollada en Node.js'; 'URL objetivo' with the value 'https://api-ventas.ejemplo.com'; and 'Repositorio' with the value 'https://github.com/empresa/api-ventas'. At the bottom of the form are two buttons: 'Registrar Aplicación' (blue) and 'Cancelar' (grey). The bottom of the sidebar shows a user profile icon and the name 'admin'.

Esta pantalla permite al usuario dar de alta nuevos proyectos dentro del framework de evaluación. Su objetivo principal es recolectar los metadatos esenciales del sistema a analizar, tales como el nombre, la descripción técnica, la URL del entorno de pruebas (testing/staging) y la ruta del repositorio de código fuente. Estos parámetros son almacenados y luego consumidos por el orquestador.

3. Interfaz de Configurar Evaluación (Asociada al CU-02: Configurar evaluación)

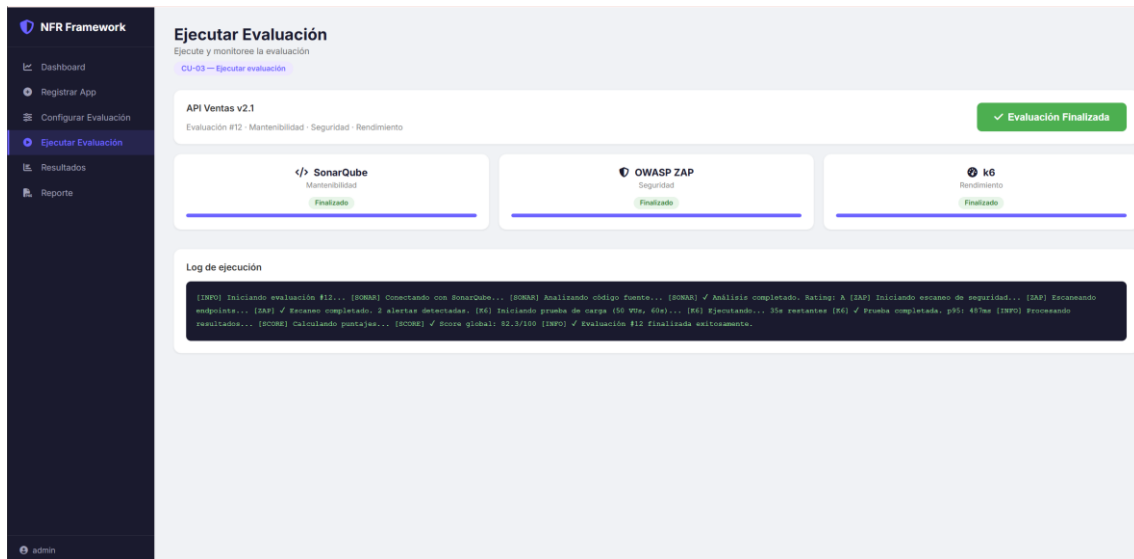
The screenshot shows the 'Configurar Evaluación' (Configure Evaluation) interface of the NFR Framework. The interface is divided into a dark sidebar on the left and a main content area on the right. The sidebar contains the following menu items: Dashboard, Registrar App, Configurar Evaluación (highlighted), Ejecutar Evaluación, Resultados, and Reporte. The main content area has a header with the title 'Configurar Evaluación' and the subtitle 'Defina los parámetros de la evaluación'. Below the header is a breadcrumb trail: 'CU-02 — Configurar evaluación'. The main form is titled 'Configurar Evaluación' and contains the following sections:

- Aplicación ***: A dropdown menu with 'API Ventas v2.1' selected.
- Categorías de evaluación ***: Three categories are listed, each with a checkbox and a button to configure the category:
 - ☒ </> Mantenibilidad **SonarQube**
 - ☒ Seguridad **OWASP ZAP**
 - ☒ Rendimiento **k6**
- Parámetros de ejecución**:
 - URL objetivo**: A text input field with 'https://api-ventas.ejemplo.com'.
 - Repositorio**: A text input field with 'https://github.com/empresa/api-ventas'.
 - Entorno**: A dropdown menu with 'Staging' selected.
 - Usuarios virtuales (k6)**: A text input field with '50'.
 - Duración (segundos)**: A text input field with '60'.
- Buttons**: A blue button with a checkmark and 'Guardar Configuración' and a grey button with 'Cancelar'.

The bottom of the sidebar shows the user 'admin'.

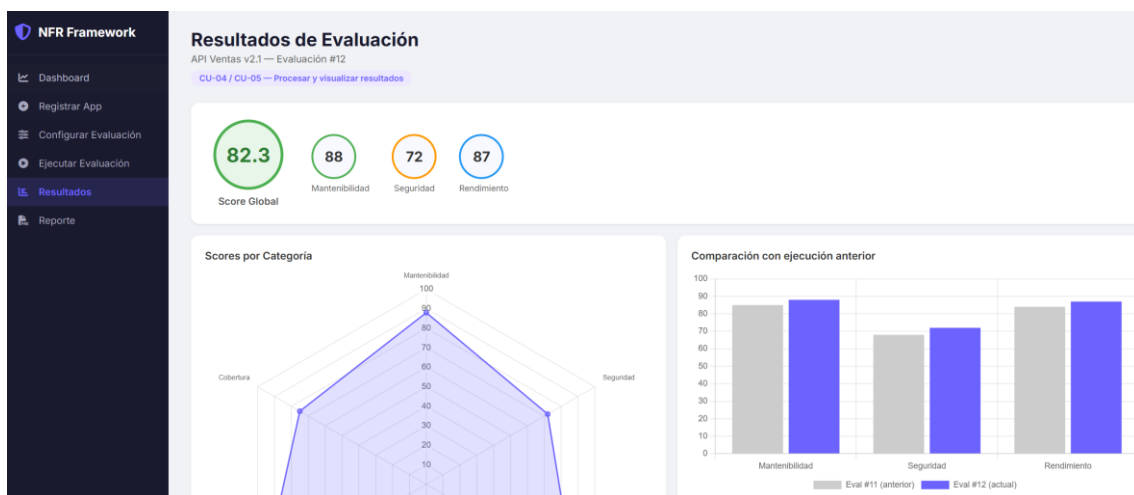
Esta interfaz tiene como función parametrizar cómo se llevará a cabo el análisis de una aplicación. Permite al usuario seleccionar activamente qué categorías de calidad desea auditar y definir qué herramientas específicas participarán en la ejecución (ej. SonarQube, k6). Desde aquí se administran las variables de entorno, la cantidad de usuarios virtuales y la duración de las pruebas.

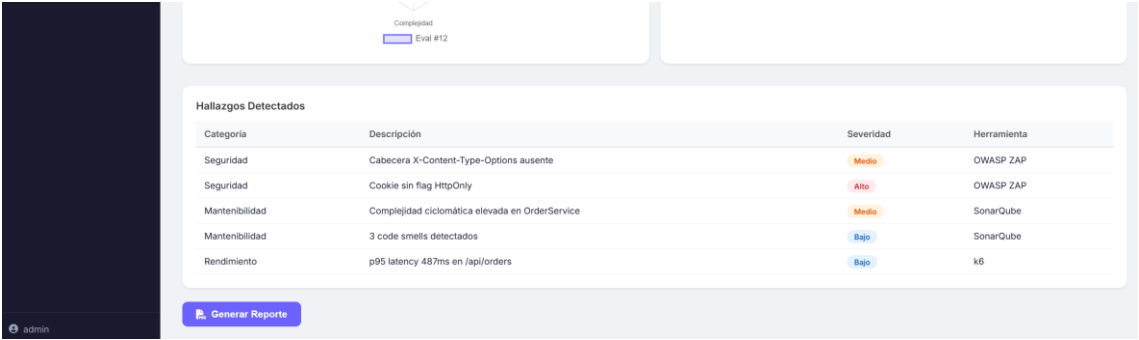
4. Interfaz de Ejecutar Evaluación (Asociada al CU-03: Ejecutar evaluación)



Esta pantalla funciona como el panel de control operativo del sistema. A través de ella, el usuario dispara el inicio del proceso de análisis y puede monitorear en tiempo real el progreso de la evaluación. La interfaz refleja el estado individual de cada herramienta orquestada, indicando si se encuentran "Pendientes", "En proceso" o "Finalizadas", e incluye una consola de logs en vivo que garantiza la trazabilidad del flujo.

5. Interfaz de Resultados de Evaluación (Asociada a los CU-04 / CU-05 — Procesar y visualizar resultados)





Esta interfaz presenta el detalle analítico de una ejecución específica. Permite al usuario visualizar un desglose interactivo del Score Global y los puntajes por categoría mediante un gráfico de radar. Además, integra un análisis comparativo automatizado contra la ejecución anterior para identificar tendencias de mejora o regresión, e incluye una tabla pormenorizada con todos los hallazgos técnicos y vulnerabilidades detectadas.

6. Interfaz de Reporte de Evaluación (Asociada al CU-06 — Generar reporte)

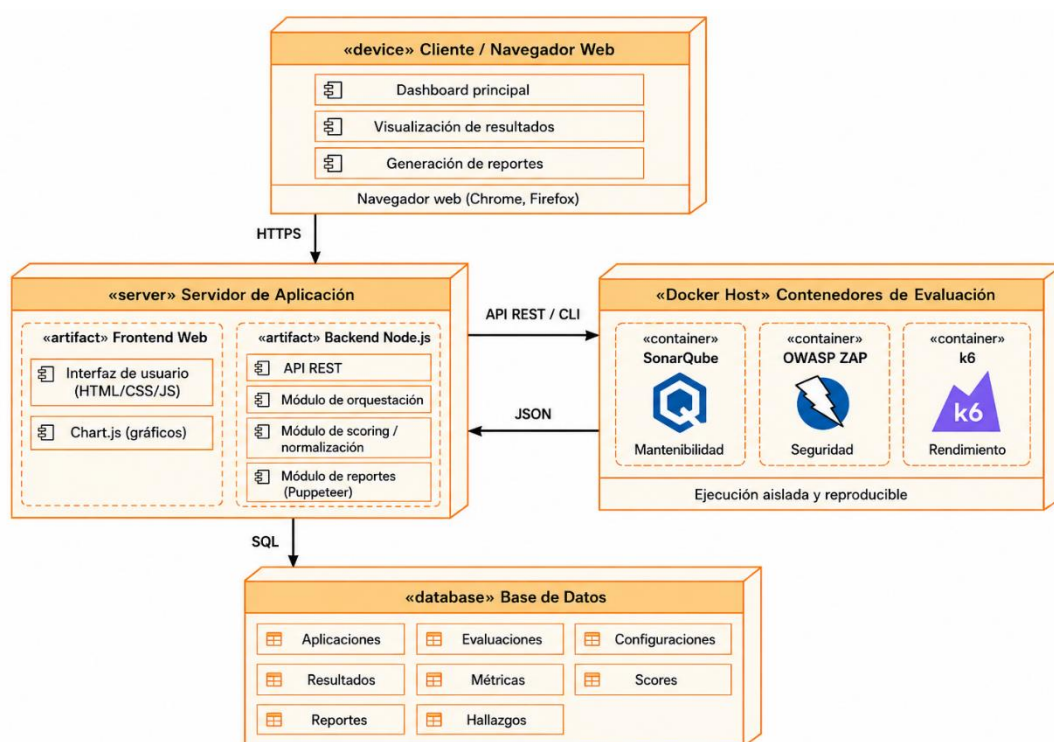


Esta pantalla tiene como función estructurar el entregable final del proceso de evaluación. Presenta un reporte estandarizado y formal, compuesto por un resumen ejecutivo, un cuadro de métricas detalladas contrastadas contra umbrales de aceptación, y

una sección de observaciones clave. Su diseño está optimizado funcionalmente para ser exportado en formato PDF o impreso, sirviendo como evidencia técnica auditable.

Diagrama de despliegue

En la siguiente figura se exhibe el diagrama de despliegue que detalla la arquitectura física del prototipo. El sistema está estructurado con un cliente web para la interacción del usuario, el cual se comunica vía APIs REST con un servidor de aplicación backend desarrollado en Node.js. A su vez, este servidor central se conecta a la base de datos para la persistencia y orquesta un entorno aislado de contenedores Docker, donde se ejecutan las distintas herramientas de análisis (SonarQube, OWASP ZAP y k6).



Seguridad

Para la seguridad del sistema, en lo relativo al acceso a la aplicación, se diseñó un mecanismo de autenticación y control de accesos basado en credenciales únicas (correo electrónico y contraseña). El correo electrónico funciona como el identificador unívoco de cada usuario en el sistema, impidiendo registros duplicados.

Las contraseñas de los usuarios deben cumplir con políticas de complejidad alineadas con las buenas prácticas de seguridad de la información. Los requisitos mínimos establecidos son:

- Una longitud mínima de ocho (8) caracteres.
- Inclusión de al menos una letra mayúscula.
- Inclusión de al menos una letra minúscula.
- Inclusión de al menos un dígito numérico.
- Inclusión de al menos un carácter especial (por ejemplo: \$, #, @, !, %, etc.).

Para proteger estas credenciales contra accesos no autorizados o fugas de información de la base de datos, las contraseñas no se almacenan en texto plano. En su lugar, se aplica una función de hashing criptográfico de un solo sentido utilizando el algoritmo bcrypt. Este algoritmo incorpora un factor de salting aleatorio y un costo computacional ajustable, lo que dificulta significativamente ataques de diccionario o de fuerza bruta mediante tablas de búsqueda precalculadas.

Adicionalmente, el sistema implementa una política de bloqueo temporal de cuentas tras cinco (5) intentos fallidos consecutivos de inicio de sesión. Esta medida mitiga de manera efectiva los intentos de acceso no autorizado mediante fuerza bruta en el formulario de login. Una vez bloqueada la cuenta, el usuario deberá realizar un proceso de recuperación de credenciales mediante un enlace enviado a su correo electrónico verificado para restablecer su acceso.

Con el fin de asegurar el principio de menor privilegio, el sistema regula los perfiles de usuario a través de un control de accesos basado en roles (RBAC). Se han definido tres perfiles específicos según las necesidades y responsabilidades de los miembros del equipo dentro de la organización:

- **Administrador:** Este perfil cuenta con privilegios totales sobre el sistema. Es el encargado de la gestión de usuarios (altas, bajas y modificaciones de cuentas), la configuración global de las herramientas externas de análisis (SonarQube,

OWASP ZAP, k6), la administración de las aplicaciones registradas en el framework, la ejecución de evaluaciones, la visualización de resultados y la generación de reportes en formato PDF.

- **Usuario Técnico:** Perfil orientado a los ingenieros de desarrollo, QA/testers y personal de DevOps. Este rol permite registrar nuevas aplicaciones de software, configurar los parámetros específicos de las evaluaciones (URLs objetivo, repositorios Git, variables de entorno), ejecutar las pruebas automatizadas de requerimientos no funcionales, consultar el historial de evaluaciones, visualizar los dashboards de resultados y generar los reportes estandarizados. No posee permisos para administrar usuarios ni alterar la configuración global del framework.
- **Usuario Consulta:** Perfil de solo lectura diseñado para el personal de gestión, líderes de proyecto o auditores externos. Permite acceder a los dashboards interactivos de resultados, consultar el historial de evaluaciones anteriores y descargar los reportes generados. Este rol tiene restringidas las acciones de modificación de configuraciones, registro de aplicaciones y ejecución de nuevas evaluaciones.

En cuanto a la política de respaldo de información, se ha establecido un esquema formal de respaldo y recuperación de datos que cubre las bases de datos de configuración y los históricos de resultados para resguardar la integridad del sistema.

En lo que respecta al resguardo y respaldo de la base de datos, para mitigar el riesgo de pérdida permanente de información debido a fallos de hardware, de red o de manipulación indebida, se mantiene un esquema de redundancia basado en tres (3) copias de seguridad de la base de datos, distribuidas en diferentes medios y ubicaciones físicas:

- **Base de datos principal (Producción):** Almacenamiento activo en el servidor donde se ejecuta el framework de evaluación, que contiene los datos en tiempo real de las aplicaciones, configuraciones, métricas obtenidas, puntuaciones de calidad e historial de ejecuciones.
- **Copia de respaldo en la nube:** Copia de seguridad automatizada que se sincroniza diariamente de forma programada durante periodos de baja actividad (00:00 horas) hacia un servicio de almacenamiento en la nube (como Amazon S3 o Google

Cloud Storage). Esta copia cuenta con cifrado en tránsito (SSL/TLS) y en reposo (AES-256) para garantizar la confidencialidad de la información.

- Copia externa (Offline): Respaldo completo que se almacena semanalmente en un dispositivo de almacenamiento físico secundario (servidor de backup local o disco duro externo) ubicado en un sector físico aislado dentro de las instalaciones de la organización. El acceso físico y lógico a este dispositivo está estrictamente restringido al personal técnico autorizado.

Esquema y retención de backups:

- Copia de seguridad incremental: Se realiza de forma automática todos los días de manera incremental para registrar los cambios diarios. Estas copias diarias tienen un periodo de retención de siete (7) días, tras lo cual se eliminan de forma automatizada.
- Copia de seguridad completa: Se realiza al final de cada semana de manera completa. Estos backups semanales se retienen por un periodo de un (1) mes antes de ser depurados.

Para garantizar la disponibilidad de la información y asegurar que el framework y los resultados de las evaluaciones estén accesibles para los usuarios cuando sea requerido, se han implementado estrategias arquitectónicas y operativas:

- Arquitectura desacoplada: La separación entre el servidor de aplicación (Node.js), el motor de base de datos y los contenedores de ejecución de herramientas externas (Docker) permite que el fallo de un componente no corrompa el almacenamiento de los datos históricos.
- Despliegue contenerizado: El uso de Docker y Docker Compose para empaquetar y ejecutar los módulos del framework y las herramientas externas (SonarQube, OWASP ZAP, k6) facilita una rápida recuperación y despliegue del entorno completo en caso de fallos del servidor físico, reduciendo el tiempo medio de recuperación (MTTR).

- Monitoreo y logs de ejecución: Se mantiene una trazabilidad completa del estado de las evaluaciones (pendiente, en proceso, finalizada o con error) y de las comunicaciones a través de las APIs REST, permitiendo a los administradores identificar cuellos de botella o fallos de conectividad en tiempo real.

Análisis De Costos

A los fines de evaluar la viabilidad económica del proyecto y estimar el presupuesto requerido para su implementación, se definieron los costos asociados al diseño, desarrollo e implementación del framework de evaluación continua. Este análisis contempla dos categorías principales: costos de desarrollo (esfuerzo de recursos humanos) y costos operativos (infraestructura tecnológica y servicios).

Para los costos operativos, se plantearon dos escenarios alternativos con el fin de proporcionar flexibilidad a la organización:

- Opción A (Infraestructura propia): Caracterizada por una mayor inversión inicial en la adquisición de hardware propio y un costo operativo mensual mínimo.
- Opción B (Servicios en la nube): Basada en un modelo de gastos operativos (OpEx) mediante el alquiler de infraestructura en la nube, minimizando la inversión inicial a cambio de un costo mensual recurrente.

Para determinar los costos de desarrollo (recursos humanos), el esfuerzo requerido para el desarrollo del framework se estimó en un periodo de tres (3) meses de trabajo. Los perfiles profesionales involucrados, sus responsabilidades principales y honorarios se detallan en la Tabla 9.

Tabla 9: Costos de desarrollo (recursos humanos)

Rol	Tareas Principales	Honorario Mensual (USD)	Dedicación (Meses)	Subtotal (USD)
Analista Funcional	Relevamiento de procesos, definición de requerimientos funcionales y diseño conceptual del modelo de scoring.	1.200	1	1.200
Desarrollador Backend	Codificación de la API REST, integración con SonarQube, ZAP y k6, procesamiento de métricas y generación de PDFs.	1.500	3	4.500
DevOps / Infraestructura	Configuración de entornos containerizados (Docker), automatización del pipeline y despliegue del framework.	1.600	2	3.200
QA / Tester	Diseño y ejecución de pruebas de calidad de software, validación funcional y de integración del framework.	1.000	2	2.000
Total Desarrollo				10.900

Nota: Los honorarios de los recursos humanos fueron estimados en dólares estadounidenses (USD) para resguardar la viabilidad financiera del presupuesto frente a la inflación. Los montos reflejan valores de mercado para perfiles de nivel Semi-Senior, tomando como parámetro base de cálculo el Valor Hora Profesional sugerido por el Consejo Profesional de Ciencias Informáticas de la Provincia de Córdoba (CPCIPC), convertido al tipo de cambio libre vigente al momento de la formulación del proyecto.

En relación a los costos operativos de la Opción A (infraestructura propia), se asume que la organización adquiere y mantiene el equipamiento físico necesario para la ejecución del sistema. Los costos de hardware y servicios se detallan en la Tabla 10.

Tabla 10: Costos operativos - Opción A (Infraestructura propia)

Recurso	Cantidad	Tareas / Propósito	Fuente de Referencia	Subtotal Inicial (USD)	Costo Mensual (USD)
Servidor Dell PowerEdge T150	1	Servidor principal para base de datos y ejecución del framework en producción.	Dell Technologies (2025)	2.000	—
Notebook Dell Inspiron (i5, 8GB RAM)	1	Equipo de desarrollo para el entorno local y mantenimiento del sistema.	Dell Store (2025)	800	—
Router Firewall Mikrotik hEX	1	Seguridad de la red perimetral y administración de accesos al servidor.	Mikrotik Catalog (2025)	150	—
Conectividad a Internet Simétrica	1	Acceso externo al servidor para integración y consultas.	Proveedor local de ISP	—	50
Total Opción A				2.950	50

Por otra parte, para los costos operativos de la Opción B (servicios en la nube), se delega el alojamiento de los servidores y bases de datos a proveedores de servicios cloud, eliminando la adquisición de servidores físicos (Tabla 11).

Tabla 11: Costos operativos - Opción B (Servicios en la nube)

Recurso	Cantidad	Tareas / Propósito	Fuente de Referencia	Subtotal Inicial (USD)	Costo Mensual (USD)
Servidor Virtual AWS EC2 (t3.medium)	1	Hosting de la API REST del framework y orquestador de evaluaciones.	AWS Calcula- tor (2025)	—	50
Base de da- tos en la nube Mon- goDB Atlas	1	Almacena- miento ad- ministrado de configu- raciones e históricos de resultados.	MongoDB Pri- cing (2025)	—	30
Almacena- miento en la nube (AWS S3)	1	Resguardo diario de ba- ckups auto- matizados del sistema.	AWS Storage (2025)	—	20
Notebook desarrollo Dell Inspi- ron	1	Estación de trabajo para manteni- miento y desarrollo local.	Dell Store (2025)	800	—
Total Op- ción B				800	100

A modo de resumen comparativo de costos, en la Tabla 12 se presenta un cuadro consolidado que compara ambas opciones operativas frente al costo de desarrollo del sistema.

Tabla 12: Cuadro comparativo de alternativas operativas

Alternativa	Inversión en Desa- rrollo (USD)	Inversión Operativa Inicial (USD)	Costo Mensual Re- currente (USD)
Opción A: Infraes- tructura propia	10.900	2.950	50

Opción B: Servicios en la nube	10.900	800	100
--------------------------------	--------	-----	-----

La comparación de alternativas muestra que la Opción B (Servicios en la nube) minimiza la barrera de entrada con una inversión inicial operativa un 72.8% menor en comparación con la adquisición de servidores físicos. Esta alternativa resulta ideal para organizaciones en etapas iniciales o proyectos medianos que busquen flexibilidad de escalado y menores costos de mantenimiento de hardware.

Por otro lado, la Opción A (Infraestructura propia) resulta económicamente atractiva para organizaciones estables que posean infraestructura de red previa o centro de datos propio, ya que a largo plazo (después de 43 meses) el menor costo operativo mensual compensa la inversión inicial en equipamiento físico.

Análisis De Riesgos

A los fines de asegurar la viabilidad del proyecto, se llevó a cabo un análisis de riesgos con el objetivo de identificar, evaluar y definir planes de mitigación para las amenazas potenciales que podrían afectar el diseño, desarrollo, despliegue o la operación del framework de evaluación continua.

En primera instancia, para la identificación de riesgos del proyecto, se procedió a analizar las amenazas específicas en función de su tipología (de proyecto, técnicos, operativos o de seguridad) y determinando su causa raíz (Tabla 13).

Tabla 13: Identificación y causas de riesgos del proyecto

ID	Tipo	Riesgo Identificado	Causa Raíz
R1	Proyecto	Estimación incorrecta de tiempos de desarrollo.	Falta de experiencia previa en la planificación y estimación temporal de proyectos de similar complejidad.

R2	Proyecto	Falta de experiencia técnica en la integración de herramientas externas de análisis.	Curva de aprendizaje empinada en la configuración avanzada y parametrización de SonarQube, OWASP ZAP y k6.
R3	Técnico	Fallas en la ejecución e integración de herramientas externas de análisis.	Dependencias del entorno de ejecución local o incompatibilidad en las llamadas a través de comandos o APIs de terceros.
R4	Técnico	Resultados inconsistentes o incompatibles entre distintas herramientas.	Heterogeneidad en los formatos de salida de datos (JSON, XML, texto plano) y disparidad en las métricas de SonarQube, ZAP y k6.
R5	Técnico	Degradación del rendimiento del sistema durante la ejecución de evaluaciones.	Alto consumo de recursos de hardware (CPU y memoria RAM) al instanciar múltiples contenedores de análisis simultáneamente.
R6	Seguridad	Accesos no autorizados al sistema o alteración de datos históricos.	Debilidades en el mecanismo de autenticación o fallas en el control de acceso basado en perfiles (roles).
R7	Operativo	Pérdida o corrupción de datos de configuración y de evaluaciones históricas.	Fallos físicos en los medios de almacenamiento o problemas de integridad en el motor de base de datos relacional.
R8	Proyecto	Cambios en los requerimientos o alcance funcional durante el desarrollo.	Desalineación inicial de los objetivos técnicos o aparición de nuevas necesidades funcionales no contempladas.

Para el análisis cualitativo y cuantitativo de los riesgos detectados, se evaluó su criticidad mediante la ponderación de dos variables fundamentales: la probabilidad de ocurrencia (expresada en un valor decimal entre 0 y 1) y el efecto o impacto del riesgo sobre el proyecto en caso de materializarse (evaluado en una escala del 1 al 5, donde 1 representa un impacto insignificante y 5 representa un impacto catastrófico).

El nivel de exposición o criticidad del riesgo se determinó mediante la siguiente ecuación: Exposición al riesgo=Probabilidad×Impacto

En la Tabla 14 se presenta la evaluación cuantitativa de los riesgos identificados, ordenados de mayor a menor según su nivel de exposición.

Tabla 14: Evaluación cuantitativa de criticidad de riesgos

ID	Riesgo	Probabilidad	Impacto	Exposición	Criticidad	Riesgo
R3	Fallas en la integración de herramientas	0,7	4	2,8	Alto	Fallas en la integración de herramientas
R5	Degradación del rendimiento del sistema	0,6	4	2,4	Alto	Degradación del rendimiento del sistema
R1	Estimación incorrecta de tiempos	0,5	4	2,0	Medio	Estimación incorrecta de tiempos
R7	Pérdida o corrupción de datos	0,3	5	1,5	Medio	Pérdida o corrupción de datos
R6	Accesos no autorizados	0,3	5	1,5	Medio	Accesos no autorizados

	al sistema					al sistema
R4	Resultados inconsistentes entre herramientas	0,5	3	1,5	Medio	Resultados inconsistentes entre herramientas
R8	Cambios en requerimientos del alcance	0,4	3	1,2	Bajo	Cambios en requerimientos del alcance
R2	Falta de experiencia técnica en integraciones	0,3	4	1,2	Bajo	Falta de experiencia técnica en integraciones

Rangos de criticidad: Bajo (menor a 1,5) | Medio (entre 1,5 y 2,3) | Alto (mayor a 2,3).

Con el fin de optimizar el esfuerzo de mitigación y enfocar la atención en las amenazas verdaderamente críticas, se aplicó el principio de Pareto (regla del 80/20) a la exposición al riesgo. Este principio establece que aproximadamente el 80% de los efectos negativos en un proyecto son causados por el 20% de los factores de riesgo (los "pocos vitales").

Al calcular la participación porcentual acumulada de la exposición (Tabla 15), se determinó qué riesgos representan la mayor fuente de exposición global del proyecto.

Tabla 15: Exposición acumulada y participación porcentual

ID	Riesgo	Exposición	Participación %	% Acumulado
R3	Fallas en la integración de herramientas	2,8	19,9%	19,9%

R5	Degradación del rendimiento del sistema	2,4	17,0%	36,9%
R1	Estimación incorrecta de tiempos	2,0	14,2%	51,1%
R7	Pérdida o corrupción de datos	1,5	10,6%	61,7%
R6	Accesos no autorizados al sistema	1,5	10,6%	72,3%
R4	Resultados inconsistentes entre herramientas	1,5	10,6%	82,9%
R8	Cambios en requerimientos del alcance	1,2	8,6%	91,5%
R2	Falta de experiencia técnica en integraciones	1,2	8,5%	100,0%
Total		14,1	100%	

Como resultado de este análisis, se identificó que los riesgos R3, R5 y R1 representan el 51.1% de la exposición total del proyecto. Por consiguiente, los planes de contingencia y esfuerzos de mitigación prioritarios de la organización se concentrarán de forma estricta en estas tres amenazas clave.

A fin de controlar y reducir el impacto potencial de los riesgos más críticos y de mayor exposición (R3, R5 y R1) identificados mediante el análisis de Pareto, se definió un plan de contingencia con un conjunto de acciones de mitigación detallados a continuación.

R3 (Fallas en la integración de herramientas externas):

- Estrategia: Mitigar.

- Acción: Diseñar una arquitectura desacoplada mediante un componente central (ToolExecutor) que aísle la llamada a cada herramienta externa en subprocesos independientes. Se implementará un sistema robusto de manejo de excepciones y registros de log por cada herramienta para facilitar el diagnóstico de errores sin interrumpir la operación general del framework.

R5 (Degradación del rendimiento del sistema):

- Estrategia: Mitigar.
- Acción: Desarrollar un despachador de tareas asincrónicas que controle la ejecución secuencial o concurrencia máxima del análisis (por ejemplo, limitar a una evaluación simultánea). Se utilizarán contenedores Docker para aislar recursos asignados a SonarQube, ZAP y k6, asegurando que no consuman la totalidad de recursos del servidor principal.

R1 (Estimación incorrecta de tiempos de desarrollo):

- Estrategia: Evitar.
- Acción: Adoptar un ciclo de desarrollo iterativo e incremental con entregas semanales controladas. Se priorizará la realización de Pruebas de Concepto (PoC) sobre la comunicación e integración con las APIs de las herramientas de análisis en las primeras dos semanas de desarrollo, permitiendo reajustar estimaciones en función del avance empírico.

Conclusiones

El desarrollo del presente Trabajo Final de Grado, titulado Modelo Automatizado de Evaluación Continua de Requerimientos No Funcionales, surgió de la necesidad de resolver la compleja validación de los requerimientos de calidad (rendimiento, seguridad y mantenibilidad); una problemática ampliamente documentada en la literatura de ingeniería de software y observada de primera mano durante mi trayectoria profesional en el área de Aseguramiento de la Calidad (QA/SDET). Frente a la falta de centralización y homogeneidad en las evaluaciones actuales de estos atributos, esta propuesta se orientó al diseño de un framework que automatizara, unificara y estandarizara su análisis dentro del ciclo de desarrollo de software.

Los objetivos planteados al inicio de esta investigación fueron alcanzados con éxito. A través del desarrollo del framework propuesto, se logró establecer un modelo de evaluación capaz de integrar y orquestar la ejecución de herramientas especializadas en la medición de mantenibilidad, seguridad y rendimiento. La validación del sistema mediante el caso de estudio demostró su aplicabilidad práctica, permitiendo realizar múltiples iteraciones de prueba y verificar el comportamiento del software de manera continua. Los resultados obtenidos confirmaron que la automatización de estas evaluaciones no solo reduce la carga operativa y el esfuerzo manual en tareas de regresión, sino que también funciona como un control continuo que asegura el cumplimiento de las metas de calidad establecidas y detecta fallos en etapas tempranas, proporcionando métricas objetivas que elevan la confiabilidad del producto antes de su despliegue.

Desde una perspectiva técnica, la pieza que demandó el mayor esfuerzo de diseño y análisis fue el desarrollo del algoritmo de scoring unificado. Debido a que cada herramienta externa genera salidas en formatos diversos y con escalas de medición muy dispares (tales como la complejidad ciclomática de código, niveles de severidad de vulnerabilidades web o percentiles de tiempos de respuesta en milisegundos), fue necesario diseñar un modelo matemático de normalización que tradujera estas métricas heterogéneas en un indicador porcentual estándar y comparable. La complejidad de esta tarea radicó en

crear un algoritmo lo suficientemente genérico y robusto que no solo resolviera las integraciones iniciales, sino que también garantizara la extensibilidad futura del framework para incorporar nuevas herramientas de análisis de manera transparente.

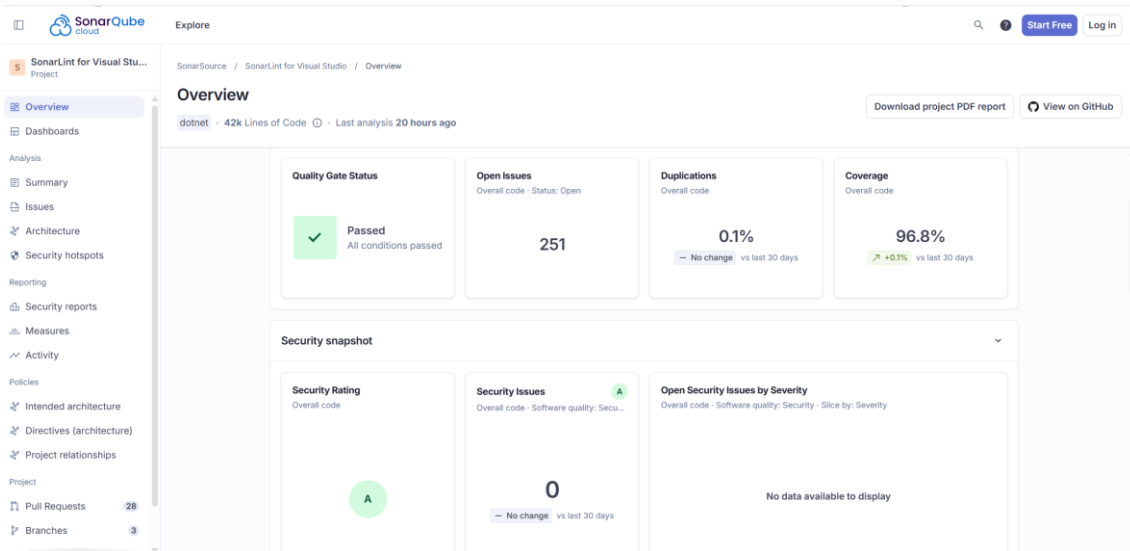
En el plano del desarrollo profesional y personal, este trabajo final representó la culminación de un ciclo de aprendizaje sumamente enriquecedor. Profesionalmente, el proyecto posibilitó plasmar más de una década de experiencia práctica en calidad de software en una herramienta concreta y escalable que, con seguridad, formará parte del repertorio técnico aplicable en futuros proyectos laborales. Asimismo, implicó profundizar en procesos de arquitectura y desarrollo de software desde cero, expandiendo las competencias más allá del testing convencional. En el ámbito personal, llevar a cabo un desarrollo de esta magnitud de manera individual, gestionando de forma autónoma los tiempos en paralelo con las exigencias del ejercicio profesional cotidiano, constituyó un desafío de alta exigencia. Haber completado con éxito este camino no solo representa la obtención del título de grado, sino la validación práctica de años de formación académica, poniendo a prueba los conocimientos adquiridos y consolidando el crecimiento integral como Ingeniero en Software.

Referencias

- Bass, L., Clements, P., y Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Boston, MA: Addison-Wesley.
- Beyer, B., Jones, C., Petoff, J., y Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. Sebastopol, CA: O'Reilly Media.
- Campbell, G., y Papapetrou, P. (2013). *SonarQube in Action*. Shelter Island, NY: Manning Publications.
- Chart.js. (2023). Chart.js documentation. <https://www.chartjs.org/>
- Chung, L., Nixon, B., Yu, E., y Mylopoulos, J. (2000). *Non-functional requirements in software engineering*. Nueva York, NY: Springer.
- Date, C. J. (2004). *An introduction to database systems* (8th ed.). Nueva York, NY: Pearson.
- Fenton, N., y Bieman, J. (2014). *Software metrics: A rigorous and practical approach* (3rd ed.). Boca Ratón, FL: CRC Press.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).
- Forsgren, N., Humble, J., y Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. Portland, OR: IT Revolution Press.
- Grafana Labs. (2023). k6 documentation. <https://k6.io/docs/>
- Google. (2023). Lighthouse. <https://developer.chrome.com/docs/lighthouse/>
- Google Developers. (2023). Puppeteer documentation. <https://pptr.dev/>
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—System and software quality models*. ISO.
- International Organization for Standardization. (2013). *ISO/IEC/IEEE 29119 software testing standard*. ISO.

- Kim, G., Humble, J., Debois, P., y Willis, J. (2016). The DevOps handbook. IT Revolution Press.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239).
- Node.js Foundation. (2023). Node.js documentation. <https://nodejs.org/>
- OWASP Foundation. (2023). OWASP ZAP documentation. <https://www.zaproxy.org/>
- Sadalage, P. J., y Fowler, M. (2012). NoSQL distilled: A brief guide to the emerging world of polyglot persistence. Upper Saddle River, NJ: Addison-Wesley.
- Sommerville, I. (2011). Software engineering (9th ed.). Boston, MA: Pearson.

Anexo A: Reporte de análisis estático de código (SonarQube)



Anexo B: Reporte de escaneo de vulnerabilidades (OWASP ZAP)

OWASP ZAP Report

Security Scan Report

Target Site: https://api-ventas.ejemplo.com

Generated on: Wed, 13 May 2026 14:32:10

Risk Summary	
Risk Level	Number of Alerts
High	1
Medium	2
Low	4

Alert Details	
High Cookie No HttpOnly Flag	
Description	A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.
URL	https://api-ventas.ejemplo.com/auth/login
Method	POST

Anexo C: Resultados de pruebas de carga y rendimiento (k6)



```

execution: local
script: load_test.js
output: -

```

```

scenarios: (100.00%) 1 scenario, 50 max VUs, 1m0s max duration

```

```

running (1m00.0s), 50/50 VUs, 15000 complete and 0 interrupted iterations

```

```

default ✓ [=====] 50 VUs 1m0s

```

```

✓ is status 200

```

```

checks.....: 100.00% ✓ 15000      X 0
data_received.....: 15 MB   250 kB/s
data_sent.....: 1.5 MB   25 kB/s
http_req_duration.....: avg=45ms  min=21ms  med=42ms  max=487ms  p(90)=85ms  p(95)=120ms
http_req_failed.....: 0.00%   ✓ 0      X 15000
vus.....: 50      min=50      max=50
vus_max.....: 50      min=50      max=50

```